

$$\sum_m \pi_{k,\alpha}^{i,m} \cdot \exp(-\sigma_{k,\alpha}^{i,m} |\mathbf{R}_{j,m}|) \quad (4)$$

where σ is a learnable 3×3 tensor parameter and π is a learnable scalar parameter. There are a total of $ikm\alpha$ copies of the σ and π parameters. m , k , i , j , and α correspond respectively to the number of nucleons, the number of determinants to be used in the ansatz, the number of orbitals, the number of electrons, and the spin of the electron taken. The term $\mathbf{R}_{j,m}$ corresponds to the 1-electron-distance vector of the j^{th} electron and m^{th} nucleon.

The envelope layer along with the computed 1-electron feature is used to model the ansatz’s orbital evaluated at a specific electron. Using the Slater-type determinants of these orbitals, the wavefunction’s value can be computed at the input electron’s position.

FermiNet Training FermiNet has 2 parts to the training: supervised pretraining and unsupervised training. During pretraining, orbital values calculated from the Hartree-Fock method are used as labels, with FermiNet’s orbital values trained to match the Hartree-Fock baseline. The loss function is:

$$\mathcal{L}_{pretrain} = \frac{1}{N} \sum_{kij} (\Phi_{kij} - \Phi_{kij}^{HF})^2 \quad (5)$$

where Φ and Φ^{HF} correspond to the calculated orbital values from the FermiNet model and the HF baseline respectively. k , i , and j correspond to the number of determinants, number of orbitals, and number of electrons respectively.

Next in the training phase, the model’s parameters are tuned according to the modified gradients using the expected energy. The parameters in the network are updated according to the following gradient

$$\nabla_{\theta} \mathcal{L}_{train} = 2\mathbb{E}_p [(E_p - \mathbb{E}_p(E_p)) \nabla_{\theta} \log |\psi|] \quad (6)$$

where $\mathbb{E}_p$ is the expected mean over the sampled electrons, E_p is the energy of sample p across all batches, and $\nabla_{\theta} \mathcal{L}_{train}$ is the modified gradient with respect to the model parameters θ , while $\nabla_{\theta} \log |\psi|$ is the gradients with respect to the $\log |\psi|$ term.

The gradients of the network are updated using the PyTorch hooks module. For the kinetic energy calculation, functorch (?) (available from Pytorch 2.0.0) is used to calculate the Hessian and the Jacobian.

Results

In Fig 1, we plot the ground state energies for Hydrogen molecules with different internuclear distances and compare them against the CCSD values calculated by PySCF. The FermiNet’s ground state energy curve closely tracks the CCSD curve. We also tested the model to estimate the LiH molecule’s ionization potential, and compared them with the CCSD and HF calculation values obtained from the NIST database (?). The result can be found in Table 1. We used a

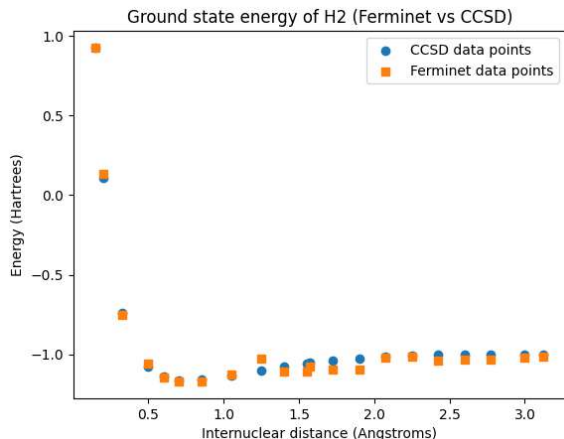


Figure 1: H2 ground state energy - FermiNet vs CCSD. Plotted using 21 different data points. The values obtained from FermiNet can be seen to closely match that of the values calculated via CCSD. Running the model with more MCMC steps and iterations can give more reliable results.

FermiNet	CCSD	HF
0.2481	0.2637	0.2387

Table 1: The ionization potential for LiH in hartrees obtained from different methods, as stated, is given. The FermiNet’s ionization potential can be seen to have improved upon the values obtained from Hartree-Fock calculations and is closer to the energy obtained using CCSD. This ionization potential was calculated using the ionic charge initialization policy that we proposed (and is not present in DeepMind’s implementation.)

batch size of 8 and 10 steps between MCMC updates to get the results. Using a larger batch size, more iterations, and more MCMC steps will decrease the noise in the results at the cost of increased training time.

Conclusion

From Figure 1 and Table 1, we observe that the model’s results closely match that of the CCSD method and improve on top of the HF baseline for LiH ionization potential calculation. We anticipate integrating FermiNet into DeepChem will lead to improved infrastructure for rapid experimentation with the FermiNet model for calculating accurate ground state energies of molecule/ion systems.

We aim to further improve the model by using Pytorch’s JIT functionality to match the capabilities of DeepMind’s JAX implementation (?). Also, we aim to integrate features from PauliNet(?), another DNN-based VMC model. PauliNet has the advantage of employing fewer parameters and achieving quicker training compared to FermiNet but with slightly lower accuracy. This is because PauliNet has “physical conditions” built into the ansatz whereas FermiNet aims to train and learn those features. Thus, incorporating physical priors into FermiNet can lead to faster conver-

gence of the model with accurate results. The standardized DeepChem implementation we have introduced will help facilitate such experimentation in future work.